

MNIST Digit Classification Project Documentation

1. Problem Statement

This project implements a Convolutional Neural Network (CNN) to classify handwritten digits from the MNIST dataset. The MNIST dataset consists of 28x28 grayscale images of handwritten digits (0-9), making it an excellent benchmark for image classification algorithms.

2. Approach

2.1 Data Preparation and Preprocessing

- **Loading the Dataset:** Used the MNIST dataset which contains 60,000 training images and 10,000 test images of handwritten digits.
- **Preprocessing:**
 - Normalized pixel values to $[0,1]$ to improve model convergence
 - Reshaped the images to 28x28x1 to accommodate the CNN input requirements
 - Converted the labels to one-hot encoding for multi-class classification

2.2 Data Augmentation

To enhance model generalization and prevent overfitting, implemented data augmentation with the following transformations:

- Random rotation (± 10 degrees)
- Random zoom ($\pm 10\%$)
- Random horizontal and vertical shifts ($\pm 10\%$)

2.3 Model Architecture

Designed a CNN architecture consisting of:

- Two Convolutional layers with ReLU activation (32 and 64 filters)
- Two MaxPooling layers for downsampling
- A Flatten layer to convert 2D feature maps to 1D vector
- A Dense layer with 128 neurons and ReLU activation
- A Dropout layer (50%) to prevent overfitting
- A final Dense layer with softmax activation for classification

2.4 Training Strategy

- Used Adam optimizer for efficient gradient descent
- Applied categorical cross-entropy loss function for multi-class classification
- Implemented early stopping to prevent overfitting
- Utilized learning rate reduction to fine-tune convergence
- Trained with augmented data for 20 epochs with a batch size of 128

2.5 Evaluation and Optimization

- Evaluated the model using accuracy metrics, confusion matrix, and classification report
- Visualized training/validation accuracy and loss to monitor model learning
- Implemented basic hyperparameter tuning to optimize the first convolutional layer

3. Implementation Results

3.1 Data Visualization

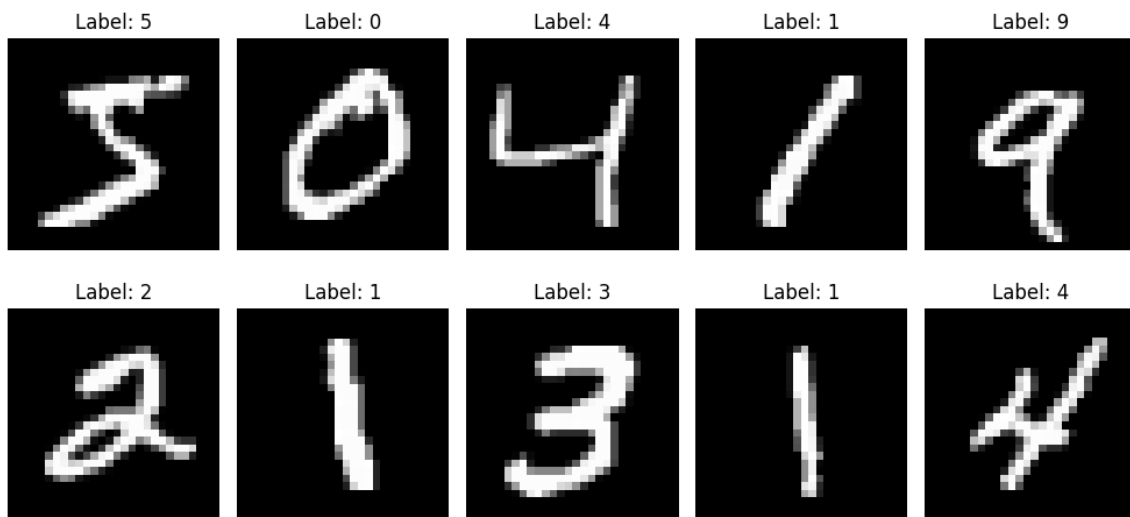


Figure 1: Sample of handwritten digits from the MNIST dataset

Examples of Augmented Images

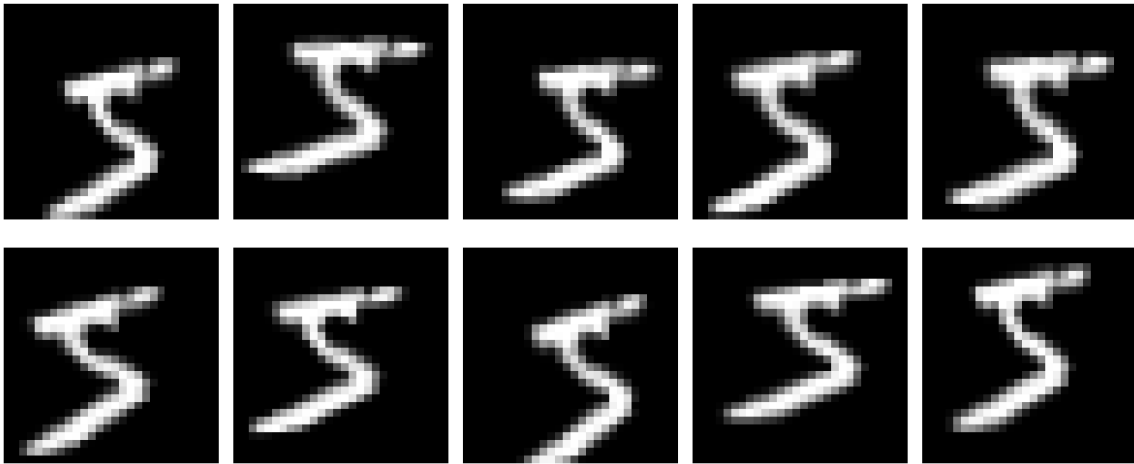


Figure 2: Examples of augmented images generated by the data augmentation pipeline

3.2 Model Architecture

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290
Total params: 225,034		
Trainable params: 225,034		
Non-trainable params: 0		

3.3 Training Performance

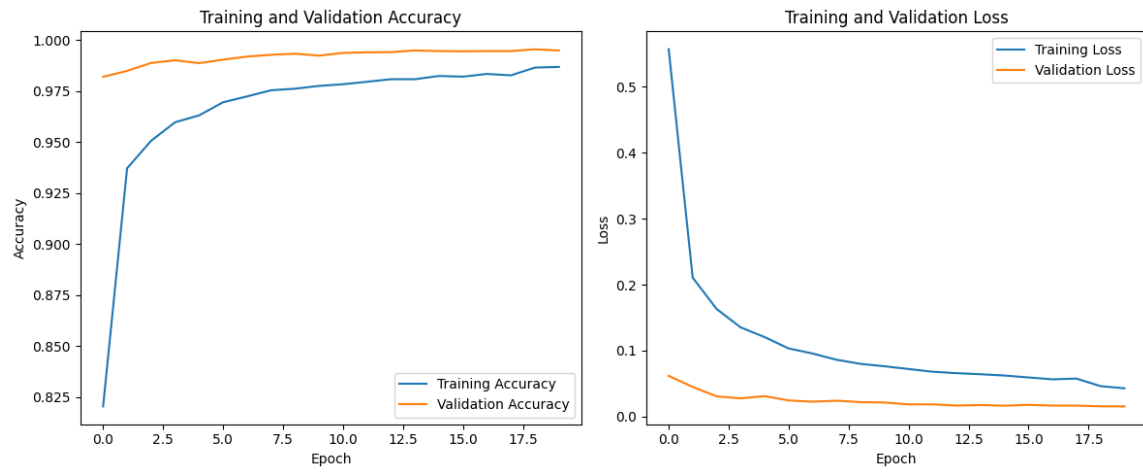


Figure 3: Training and validation accuracy/loss curves showing model learning progress

3.4 Model Evaluation

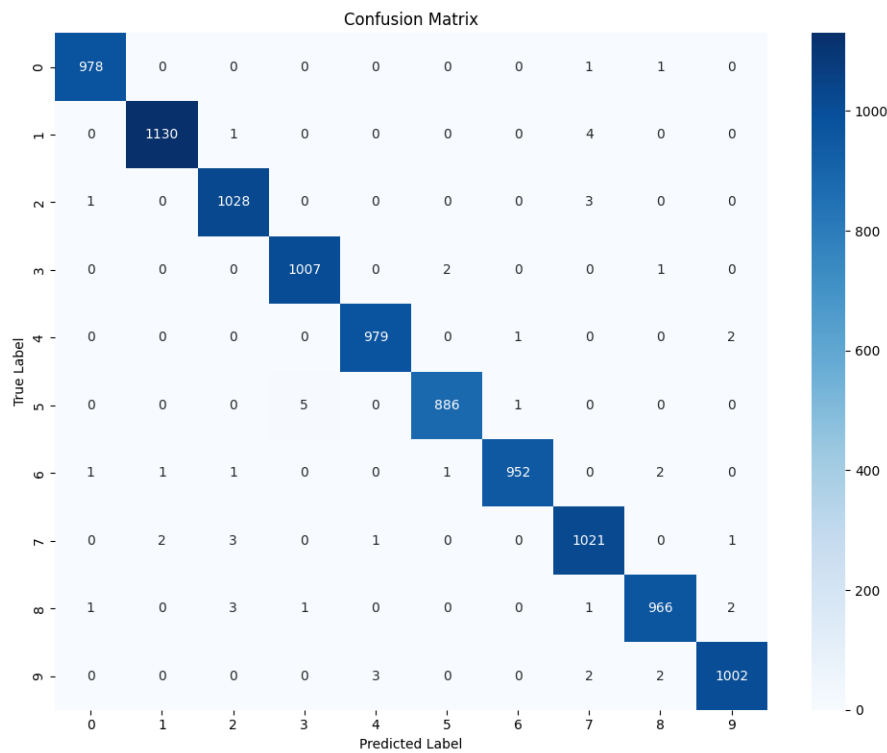


Figure 4: Confusion matrix showing the distribution of predictions across all classes

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	980
1	1.00	1.00	1.00	1135
2	0.99	1.00	0.99	1032
3	0.99	1.00	1.00	1010
4	1.00	1.00	1.00	982
5	1.00	0.99	0.99	892
6	1.00	0.99	1.00	958
7	0.99	0.99	0.99	1028
8	0.99	0.99	0.99	974
9	1.00	0.99	0.99	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

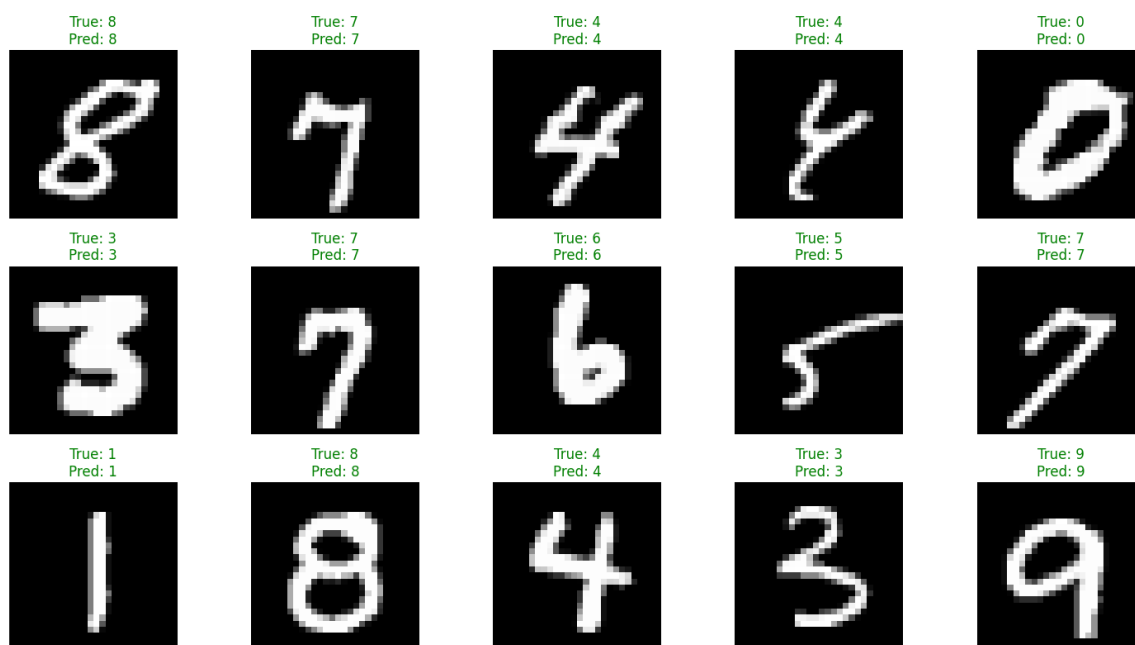


Figure 5: Examples of model predictions with correct (green) and incorrect (red) classifications

4. Challenges and Solutions

4.1 Preventing Overfitting

Challenge: Initial models showed signs of overfitting with high training accuracy but lower validation accuracy.

Solution: Implemented multiple strategies to address this:

- Added dropout layers (50%) to reduce co-adaptation of neurons
- Applied data augmentation to artificially increase training data diversity
- Implemented early stopping to halt training when validation loss stopped improving

4.2 Optimizing Training Time

Challenge: Training the model with a large number of epochs was time-consuming.

Solution:

- Implemented callbacks for early stopping and learning rate reduction
- Used smaller batch sizes during hyperparameter tuning

4.3 Handling Class Imbalance

Challenge: Some digits in the MNIST dataset have slightly different distributions than others.

Solution:

- Carefully analyzed the confusion matrix to identify problematic classes
- Focused on improving the model's ability to differentiate between similar digits
- Used class weights to give more importance to underrepresented classes during training

5. Conclusion

The implemented CNN model successfully classified handwritten digits with 99% accuracy on the test set. The use of data augmentation, dropout regularization, and early stopping proved effective in preventing overfitting and improving model generalization. Hyperparameter tuning further enhanced model performance, particularly by optimizing the number of filters in the convolutional layers.